## Getting Started Modules

- The subjects outlined in these modules are not web development topics, but they are useful for anyone wanting to learn front-end web development to have an understanding of
- We don't consider learning these topics as required before moving on to the core modules, hence we haven't used the word 'prerequisite'.
- However, we believe students will have an easier time if they spend some time on these topics first
    1. Soft Skills
    2. Environment Setup

## Soft Skills

- This module provides recommendations of soft skills that students can aim to get better at while learning web development, and which constitute good traits to have when entering the industry
- They will help immensely in developing the right attitudes for learning, researching, and collaborating, and increase the chances of getting hired
- Recommended soft-skills are listed below –
    1. Constant learning mindset
    2. Open to embracing failure
    3. Effective research
    4. Collaboration and teamwork
    5. Succeeding in job interviews
    6. Workflows and processes
    7. Relevant contextual information

## Environment Setup

- This module includes topics related to the setup and usage of the computer system that you will use to implement websites/apps
- These topics are not directly related to creating web code, but you will benefit greatly from understanding the operating system you are working with
- Topics are related to environment-setup are listed below –
    1. Computer basics
    2. File systems
    3. Browsing the web
    4. Command line basics
    5. Code editors

## Soft Skills (Constant Learning Mindset)

- Student should get into the mindset of constant learning
- The web is constantly evolving and technologies and trends are always changing, and they need to constantly update their knowledge to keep up
    1. Get into the habit of regularly reading technical news, blogs, browser release notes
    2. Engage in reading tasks or small research projects semi-regularly
    3. Set aside specific learning time to spend on acquiring new skills
    4. Be curious
- Recommended news and information sites
  (MDN | CSS Tricks | Dev | freeCodeCamp | codeCademy)

## Soft Skills (Open to Embracing Failure)

- A very common issue that causes students and new developers to shy away from experimentation and taking risks (for example when starting new projects or exploring new ideas) is fear of failure
- Spend some time learning about the value that can be gleaned from making mistakes, and the lessons that can be learned and applied in the future in similar situations
- Here are some tips to improve this skill –
    1. Define a safe space/peer group where people are free to ask questions and failure will not be judged harshly
    2. Look to your local community and try to find meetup groups with people who can either give you help and advice or are facing the same issues you are and can provide moral support or experiment together with you
    3. (For educators) Set up the marking schemes for your assessments so that you can still get a reasonable number of marks even if you didn't get the correct result provided the process is well documented. Award extra marks for innovation
    4. Run show 'n' tell or one-on-one sessions part-way through a project with peers and mentors to get feedback and insights into where you are going wrong and get advice on how to get back on the right path
    5. Run retrospective meetings to analyze projects, look at what didn't go so well, and talk about how to improve things next time

Soft Skills (Effective Research)

- Developers spend a lot of time searching for solutions to problems encountered in work.
- Students should learn effective strategies for finding answers, and where to use which methods (don't' jump straight to pestering the senior dev every time you hit a roadblock)
- These strategies include –
    1. Consulting the documentation
    2. When stuck with using a tool/product, consult the official documentation first
    3. It is surprising how many people do not think to do this; official docs are often geared towards beginners, so experience people may not think of them as being suitable
    4. Learn about different types of documentation – for example, tutorials, references, and other types – and understand when each type is useful
        - Using search engines effectively
        - Choosing valid information sources
    5. Choose recommended sites such as (Stack Overflow and MDN)
    6. Consider whether the techniques discussed are out-of-date
    7. Example, does an article on CSS talk about modern approaches like grid and flexbox, or does it still present obsolete techniques like multi-column layouts with floats
    8. Does it still talk about hacks for ancient browsers like internet Explorer or Netscape4
        - Using social media effectively
    9. Build a network of folks who can help
    10. Join community groups where you can look for answers.
    11. Example (MDN Web Docs Community on Discord | freeCodeCamp | CodeNewbie)
    12. Learn to give back as well as take; web developers who reciprocate are much more likely to build strong relationships and keep getting help
    13. When you find a useful answer, write about it
    14. Example, blog about it or share it on a social network
    15. Not only will the process of writing clarify the concepts to you, but you'll also get validation and/or feedback from the community
    16. it could also help you to start making a name for yourself in the industry
        - Making effective use of an experienced contact's time
    17. By 'experienced contact', we mean a teacher, mentor, or senior developer
    18. Ask them what communication methods they prefer
    19. Think about what questions to ask them before, consider time limited and precious
    20. Be sure to do some background research about the topic beforehand and don't ask questions that you can find answers to by searching web or official documentation
    21. Timebox the session to say 30 minutes
    22. Prioritize your issues
    23. Set a goal for the session, for example, 'try to find a solution to the highest priority issue'; solving the biggest issue may also lead to a fix for other issues
        - Rubber ducking as an effective help mechanism
        - Using AI to help with coding issues (ChatGPT or Copilot). You should use AI tools with caution, and familiarize yourself with their strengths and weakness
    24. On the plus side, they can speed up research/searches enormously, and help with structuring code and copy
    25. On the other hand, AI tools have no reasoning skills and frequently provide answers that are misleading or just plain wrong.
    26. Shouldn't assume that AI answers are correct, and test them with other sources
    27. Notes – There is definitely a balance to knowing the right time to ask for help.
    28. Notes – developers shouldn't constantly pester their peers/colleagues, equally, they shouldn't soldier on and pretend they know what they are doing when they don't
    29. Notes – Consider the value of saying 'I don't know' at the right time

Soft Skills (Collaboration and Teamwork)

- As a professional in the web industry, you are going to have to work with other people on projects, and while brainstorming ideas and proposals
- Not everyone is born with an innate ability to work in a term, so it is beneficial to start incorporating some best practices early on and putting work into areas where you think you are lacking
- Recommendations –
    1. Learn about empathy, humility, conflict resolution, and cooperation.
       In all engagements, stay polite and respectful and do not use offensive language
    2. While working in team in real world, will frequently be expected to do peer reviews.
       Practice how to deliver feedback constructively and respectfully.
       When receiving feedback, practice how to not take it personally, and focus on the positives and what you can learn.
    3. Participate in pair programming, or work in teams on assessments to experience working with other people
    4. Practice running a real software project, with a timeline, plan, and responsibilities.
       Learn about the software development lifecycle.
       Pick up some basic project planning skills and tools to be able to estimate and plane your work/project
    5. As part of the course, blog about your work, learnings, and roadblocks, share your code repositories, get peers to critique your work, and offer updates to fix issues in other people's work
    6. Join a slack channel, discord, or a similar space, ask peers for help, share resources, and discuss the work to be done (for example, Frontend Developers Discord Server)
    7. Practice asking and answering questions.
       Even if they seem somewhat trivial, always come up with one or two questions to ask when discussing or reviewing peer work.
       It is essential to practice explaining what you are doing and asking the right questions to find out what you need to know.
    8. Help each other, rather than waiting for a teacher or senior dev to go around.
       Less able peers will get help more quickly, and mere able peers will become mentors and experience the satisfaction that it brings
    9. Observe and learn from other experienced folks how be engage in discussions as well as how to approach problem-solving/debugging
    10. Join an open-source project to practice the skills you learn, engage with folks in the community, and learn from observing others
        (See, How to Contribute to Open Source Project – A Beginner's Guide, for useful info)

## Soft Skills (Succeeding in Job Interviews)

- Technical job interviews can be very demanding, and some have quite specific requirements
- Recommendation –
    1. Learn effective strategies for job searching. For example –
        - Attend networking events and job fairs to meet potential employers
        - Keep an inventory of the people you meet and the companies you apply to
        - Follow up with any promising leads you meet
    2. Create a portfolio
    3. Build the perfect resume
    4. Get experience – build real projects and contribute to open source
    5. Build your online persona
    6. Use sites like LinkedIn to help with the above
    7. Practice writing answers for coding and design interview questions
    8. Build a collection of anecdotes to use for experience – based interview questions
    9. Be aware of the attributes that hiring managers look for in a candidate and prepare accordingly –
        - Someone they can get along with
        - Positive attitude, respectful, empathic, constructive
        - Open-minded and works well in a diverse team with diverse viewpoints
        - Graceful when a decision does not go their way, able to align for greater good
        - Good communicator and relationship builder
        - Tenacious, focused, good problem solver
        - Having a good portfolio
    10. Be patient. Even the best candidates will get rejections from multiple job applications before they land the job they want
- Resources –
    1. Technical Interviewing 101: Ultimate Guide to Acing Your Tech Interview
    2. 30 Technical Interview Questions and Tips for Answering

## Soft Skills (Workflows and Processes)

- An important aspect of technical projects that beginners often miss out on is an ideal of the bigger picture of a project
- They might learn an individual tool or language and understand what they need to do, but be unaware of all the other codebases, tools, systems, and job roles that go together to deliver an entire web application
- It is useful to understand the following at a high level –
    1. Typical technology combinations and applications architectures in common projects
    2. Typical processes for a technical project, including where different tools are used in those processes
    3. Typical job roles, and where they are involved in those processes
    4. Common work management styles, such as agile and waterfall
- Resources –
    1. What is a Tech Stack and How Do They Work
    2. Website development team structure: roles and processes
    3. Waterfall vs. Agile vs. Kanban vs. Scrum: What’s the difference

Soft Skills (Relevant Contextual Information)

- While not essential for understanding the technical topics listed in the curriculum, there is a range of contextual information that can help developers gain a well-rounded and flexible perspective.
- Recommendations –
    1. To understand why things are the way they are, study the relevant historical context. For example –
        - Why was the web designed like it is in terms of data delivery when arguably faster mechanisms exist
        - Why does the web use a document model with links as a central feature when these days it is largely used to build apps
        - Why are web standards created like they are, in collaboration, when the process isn't necessarily as efficient as it could be?
    2. Study general programming concepts where relevant, for example –
        - The purpose of objects, and what they enable in terms of the design of a language like JavaScript and its surrounding APIs
        - How loops work and why they are needed